

Bitte beantworten Sie die Fragen in den dafür vorgesehenen Feldern. Wenn Sie keinen Platz mehr für eine Antwort haben, fahren Sie bitte auf der Rückseite fort und geben an wo es weiter geht.
Abgesehen von Internetfähigen Geräten sind alle Hilfsmittel zugelassen.
Schreiben Sie Ihre Klausur bitte nicht mit Bleistift und lassen Sie die Klausur bitte geheftet.
Sie haben 90 Minuten Zeit. Mit 43 Punkten gilt die Klausur als bestanden.

Vor- und Nachname (in Druckbuchstaben): _____

Matrikelnummer und Studiengang: _____

Aufgabe Nr.:	Punktzahl:	Davon erreicht:
1	5	
2	9	
3	8	
4	7	
5	8	
6	9	
7	10	
8	6	
9	5	
10	9	
11	14	
Summe	90	

Note und Unterschrift des Erstprüfers: _____

Unterschrift des Zweitprüfers: _____

Lernraum I

1. (5 Punkte) Geben Sie die Laufzeit für die folgende Funktion `fib` in der O -, Ω - und (falls vorhanden) Θ -Notation an. Es sollte ersichtlich sein wie Sie auf diese asymptotischen Laufzeiten gekommen sind (bspw. sehr kurze Begründung). Führen Sie bei Bedarf eine Worst- und Best-Case Analyse durch.

```
def fib(n):  
    if 0 <= n < 5:  
        return fib(n-1) + fib(n-2)  
    else:  
        print("hello world")  
        return 0
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Lernraum II

2. Eine Hashtabelle T speichert Benutzerkonten an der TH Köln. Der Schlüssel `key` ist eine E-Mail-Adresse und der Wert `value` der Name des Nutzers. Gegeben sind folgende Funktionen, die ein Schlüssel-Wert-Paar in T einfügen und finden.

```
function insert(T, key, value):
    i = hash(key)
    T[i].append((key, value))

function find(T, key):
    i = hash(key)
    for (k, v) in T[i]:
        if k == key:
            return v
    return None
```

Angenommen, n Schlüssel wurden bereits eingefügt und die Hashtabelle hat m Buckets.

- (a) (2 Punkte) Welche Technik wird bei den implementierten Funktionen zur Auflösung von Hashkollisionen genutzt?

.....

- (b) (4 Punkte) Was ist die Best und Worst Case Laufzeit von `find` in der Θ -Notation? Begründen Sie.

.....
.....
.....
.....
.....

- (c) (3 Punkte) Welche der folgenden Hashfunktionen ist für diese Anwendung am besten geeignet? Kreuzen Sie eine Antwort an und begründen Sie Ihre Wahl kurz (evtl. über das Ausschlussprinzip).

A. `hash(key):`
 `h = 0`
 `for c in key:`
 `h += ord(c)`
 `return h mod random(1, m)`

B. `hash(key):`
 `domain = key.split("@")[1] # domain wäre bspw.: th-koeln.de`
 `return ord(domain[0]) mod m`

C. `hash(key):`
 `h = 0`
 `for c in key:`
 `h = (h * 31 + ord(c)) mod m`
 `return h`

.....
.....
.....
.....

-
-
-
-

-
- A full-page sheet of white graph paper with a light gray grid. The grid consists of small squares, approximately 10 units wide by 10 units high. There are no margins or additional markings on the page.

-
-
-
-

Lernraum III

4. In einem Multiplayer-Online-Spiel mit potenziell Millionen Spielern können sich Spieler zu Allianzen zusammenschließen. Allianzen können miteinander fusionieren, sodass größere Allianzen entstehen. Für das Spiel ist es wichtig folgende Abfragen effizient beantworten zu können:

1. zu welcher Allianz ein bestimmter Spieler gehört und
2. ob zwei Spieler zur selben Allianz gehören.

(a) (2 Punkte) Welchen abstrakten Datentyp (ADT) würden Sie verwenden, um diese Abfragen effizient zu unterstützen?

.....
.....

(b) (2 Punkte) Wie werden Spieler und Allianzen in diesem ADT modelliert?

.....
.....

(c) (3 Punkte) Welche grundlegende(n) Operation(en) muss der ADT bereitstellen, um die zwei oben beschriebenen Abfragen zu ermöglichen? Beschreiben Sie in einem Satz was die Operation(en) macht/machen?

.....
.....
.....
.....
.....
.....

5. (8 Punkte) Modifizieren Sie den DFS-Algorithmus so, dass er eine Liste aller vom Startknoten s erreichbaren Knoten zurückliefert. Die Reihenfolge der Knoten in der Liste spielt keine Rolle. Verwenden Sie Pseudocode.

Geben Sie an, an welchen Stellen im Code Änderungen oder Ergänzungen vorgenommen werden müssen. Es reicht, die betreffenden Zeilennummern zu nennen und den jeweils hinzugefügten oder geänderten Code in Pseudocode (oder einer Sprache Ihrer Wahl) anzugeben.

Algorithm 1 Iterative Tiefensuche (DFS)

```
1: function dfs(Graph graph, Vertex start) {
2:   Stack<Vertex> perimeter = new Stack<>();
3:   Set<Vertex> visited = new Set<>();
4:
5:   perimeter.add(start);
6:
7:   while (!perimeter.isEmpty()) {
8:     Vertex from = perimeter.remove();
9:     if (!visited.contains(from)) {
10:      for (Edge edge : graph.edgesFrom(from)) {
11:        Vertex to = edge.to();
12:        if (!visited.contains(to))
13:          perimeter.add(to);
14:      }
15:      visited.add(from);
16:    }
17:  }
18: }
```

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

6. Ein:e Trainer:in einer Mannschaftssportart (z. B. Fußball oder Handball) möchte analysieren, wie häufig Pässe zwischen den einzelnen Mitspielern gespielt werden. Dazu soll das Passverhalten der Mannschaft als Graph modelliert und gespeichert werden.

Beantworten Sie die folgenden Fragen:

- (a) (1 Punkt) Was stellen die Knoten des Graphen dar?

.....
.....

- (b) (2 Punkte) Was stellen die Kanten des Graphen dar? Sind die Kanten gerichtet oder ungerichtet? Begründen Sie kurz.

.....
.....
.....

- (c) (3 Punkte) Sind die Kanten gewichtet oder ungewichtet? Begründen Sie Ihre Antwort.

.....
.....
.....
.....

- (d) (3 Punkte) Sollte der Graph als Adjazenzmatrix oder als Adjazenliste implementiert werden? Begründen Sie Ihre Entscheidung. Gehen Sie davon aus, dass jeder Spieler mindestens einen Pass zu jedem seiner Mitspieler spielt.

.....
.....
.....
.....
.....

7. Ein Online-Ticketanbieter verwaltet Veranstaltungen anhand ihres Startzeitpunkts (Datum und Uhrzeit). Es sollen regelmäßig:

- Bereichsabfragen durchgeführt werden, d.h. nach allen Veranstaltungen gesucht werden, die in einem bestimmten Zeitintervall liegen (z. B. nächsten Samstag zwischen 14:00 und 22:00 Uhr),
- die nächste bevorstehende Veranstaltung bestimmt werden (bspw. heute Abend),
- neue Veranstaltungen eingefügt werden.

Beantworten Sie die folgenden Fragen:

(a) (4 Punkte) Welcher abstrakte Datentyp und welche Datenstruktur eignen sich für diese Anwendung am besten? Begründen Sie Ihre Wahl.

.....
.....
.....
.....
.....
.....

(b) (3 Punkte) Welche Laufzeiten haben (1) **Insert**, (2) **Search** (sucht eine Veranstaltung gegeben ein Datum inkl. Uhrzeit) und (3) eine Bereichsabfrage in dieser Datenstruktur im Worst Case?

.....
.....
.....
.....

(c) (3 Punkte) Wäre eine Hashtabelle für diese Anwendung geeignet? Begründen Sie kurz.

.....
.....
.....

8. Ordnen Sie jeder der folgenden Anwendungen einen passenden Algorithmus zu. Wählen Sie aus den folgenden Algorithmen:

- Dijkstra's Algorithmus
- Breitensuche
- Erreichbarkeitsproblem
- Topologische Sortierung
- Insertionsort
- Memoization
- Prims Algorithmus
- Dynamische Programmierung
- Heapsort

(a) (2 Punkte) Eine Paketverteilzentrale möchte feststellen, über wie viele Zwischenstationen ein Paket bis zum Empfänger mindestens transportiert werden muss, wenn alle Transportwege gleichwertig sind.

.....

(b) (2 Punkte) Ein Navigationssystem berechnet für Autofahrer die schnellste Route, wobei Straßen unterschiedliche Fahrzeiten besitzen.

.....

(c) (2 Punkte) Ein Build-System muss Quellcodedateien in einer Reihenfolge kompilieren, sodass alle Abhängigkeiten eingehalten werden.

.....

9. (5 Punkte) Zeichnen Sie die Datenstruktur, die bei Quick-Find genutzt wird und genau die gleiche Situation modelliert wie die drei dargestellten Up-Trees aus der Quick-Union Implementierung.

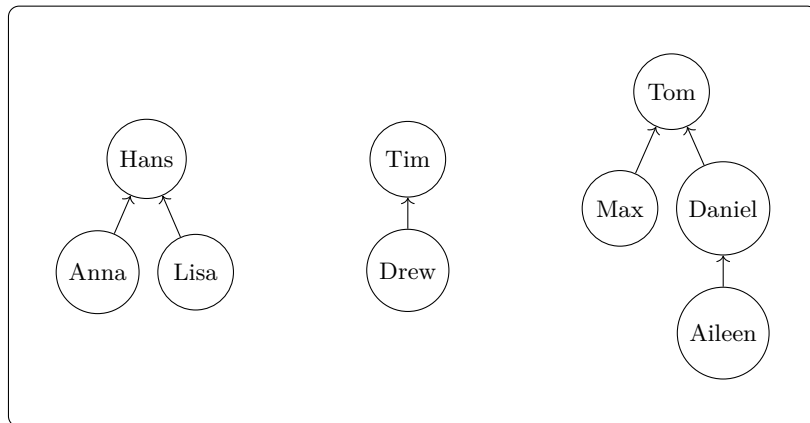
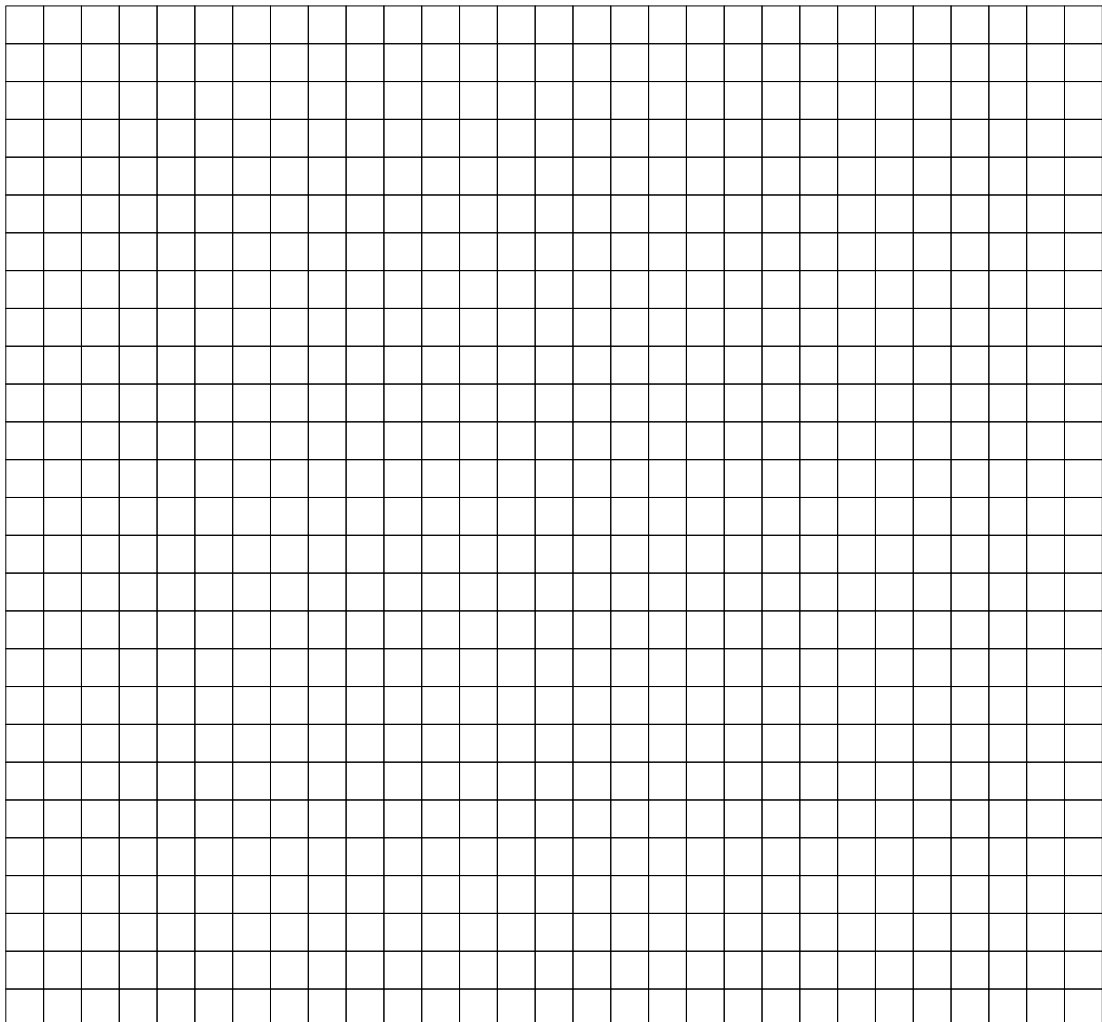


Abbildung 1: Darstellung von drei disjunkten Mengen.



Lernraum IV

10. Wählen Sie für jedes der folgenden Szenarien den am besten passenden Sortieralgorithmus aus und begründen Sie Ihre Wahl in einem kurzen Satz.

Zur Wahl stehen: Insertionsort, Selectionsort, Mergesort, Heapsort, Bucketsort

- (a) (3 Punkte) Ein Online-Shop zeigt seinen Kund:innen eine Rangliste der aktuell meistverkauften Produkte. Die Liste wird sehr häufig aktualisiert, wobei sich die Verkaufszahlen zwischen zwei Aktualisierungen nur geringfügig ändern.

.....

.....

.....

.....

.....

.....

.....

- (b) (3 Punkte) Ein Messsystem erfasst jede Sekunde die Luftfeuchtigkeit an Millionen von Sensoren. Die Messwerte liegen ganzzahlig zwischen 0 % und 100 %. Die Werte sollen regelmäßig sortiert werden, um Extremwerte schnell zu erkennen.

.....

.....

.....

.....

.....

.....

.....

- (c) (3 Punkte) Ein Datenanalyse-Tool sortiert eine große, vollständig unsortierte Liste von Datensätzen nach einem numerischen Score. Die Sortierung muss auch im Worst Case asymptotisch schnell sein, zusätzlicher Speicher soll möglichst vermieden werden.

.....

.....

.....

.....

.....

.....

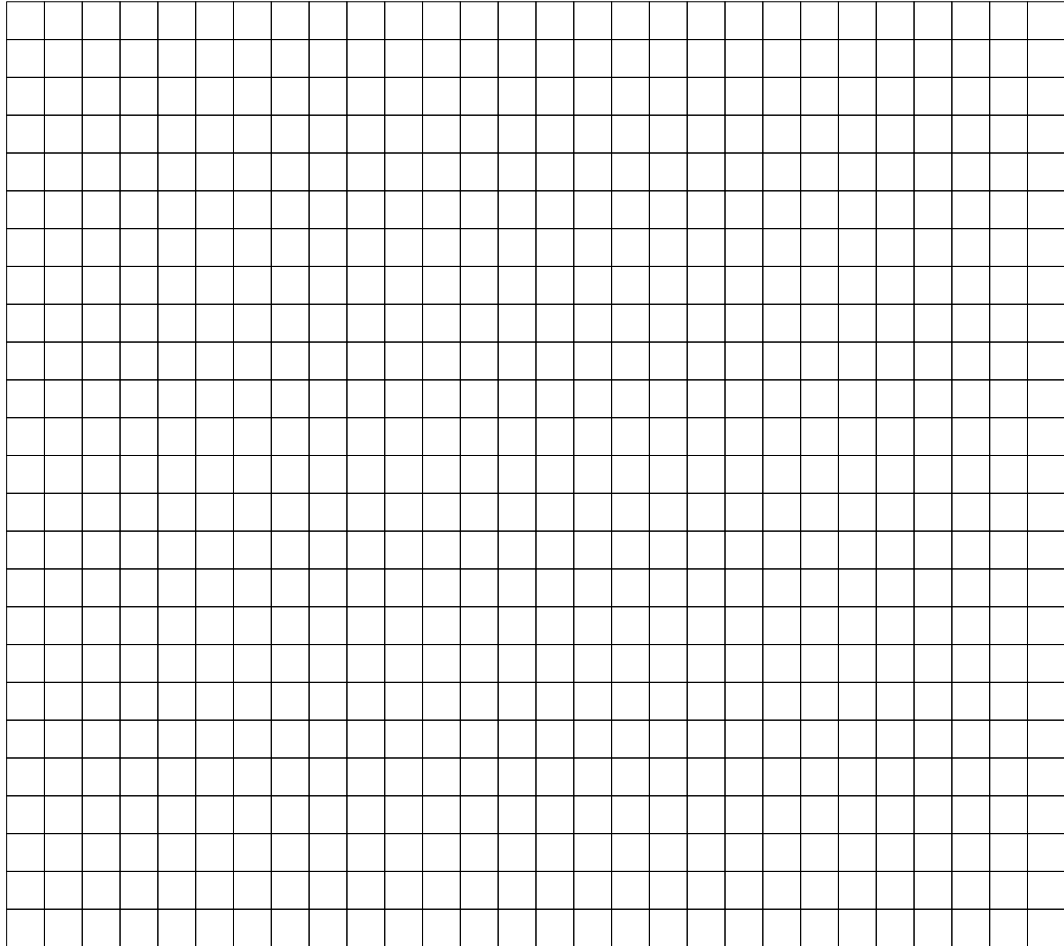
.....

11. Rekursion und Rekursionsbaum

Gegeben sei folgende rekursive Funktion:

```
function f(n):  
    if n <= 1:  
        return 1  
    return f(n-1) + f(n-1)
```

- (a) (6 Punkte) Zeichnen Sie den Rekursionsbaum für $f(4)$.



- (b) (2 Punkte) Bestimmen Sie die Laufzeit in der Θ -Notation.

.....

.....

.....

.....

- (c) (6 Punkte) Liefert die folgende Funktion `fnew` für `n=4` das gleiche Ergebnis wie `f(4)` zurück? Begründen Sie Ihre Antwort, bspw. durch eine Rechnung.

```
function fnew(n):  
    if n <= 1:  
        return 1  
    return 2 * fnew(n-1)
```

